

Bölüm 8: Ana Bellek





Bölüm 8: Ana Bellek

- Arka Plan Bilgisi
- Yer Değiştirme (Swapping)
- Bitişik Bellek Yerleşimi (Contiguous Memory Allocation)
- Sayfalama (Paging)
- Sayfa Tablosunun (Page Table) Yapısı
- Bölütleme (Segmentation)
- Örnek: Intel Pentium

Yiğit Emre Yönlük
A1-905094 öğreniyorum.net





Hedefler

- Sınırlı hafızayı verimli kullanma yöntemleri
 - **İşlemleri ne kadar düzgün hafızaya yerleştirirsek o kadar verim alırız**
 - Sayfalama (paging) ve bölütleme (segmentation) hafıza Yönetimi
- İşlemi belleğe almak için yeterli hafıza var mı?
 - Belleği verimli kullanmak için işlemleri nasıl yerleştirmeliyiz?
 - Mantıksal-fiziksel adres dönüşümleri





Arka Plan Bilgisi

- Çalıştırılmak istenen program öncelikle diskten belleğe alınmalı ve bir **işleme dönüştürülmelidir**.
İşlem bellekteyken belleğin tümünü onun kullandığını sanar, bir mantıksal adresi vardır/fiziksel adres
- CPU'nun direk olarak erişebileceği kayıt birimleri yalnızca ana bellek (main memory) ve yazmaçlardır (registers)
- CPU'nun yazmaçlara erişimi bir CPU birim zamanı (veya daha az) sürer
- Ana belleğe erişim pek çok CPU birim zamanı sürebilir
- **Ön bellek (cache)** ana bellek ile CPU yazmaçları arasında yer almaktadır
- Ana belleğin korunması sistemin doğru çalışması için şarttır

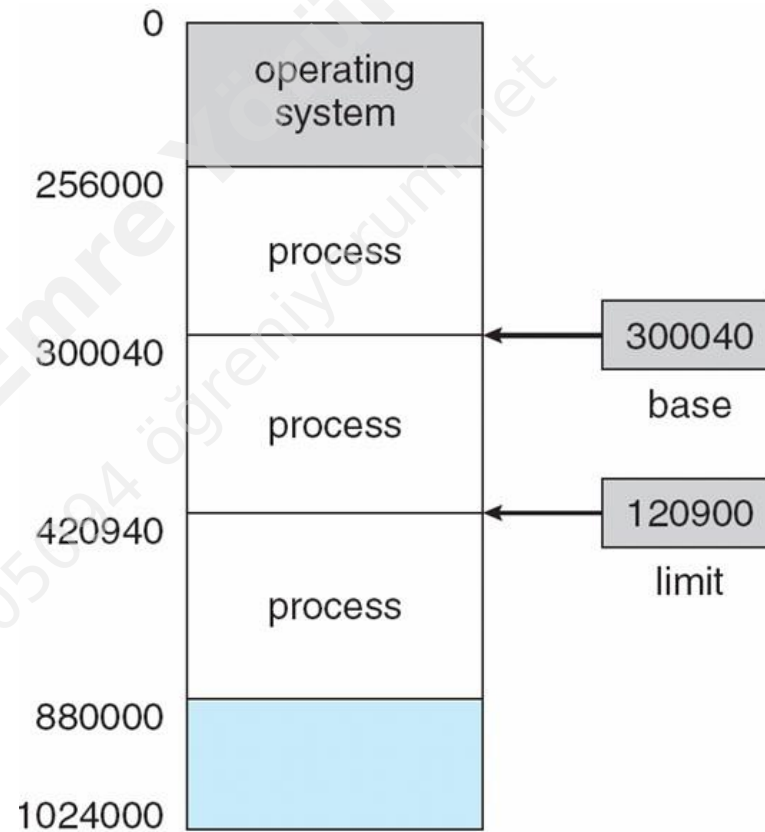




Taban ve Sınır Yazmaçları

- **Taban (base)** ve **sınır (limit)** yazmaçları mantıksal adres uzayını tanımlar

-Mavi alan boş alan
-Başlangıç-base
-Limit-max erişebileceği yer
-Process-0
-Erişmek istediği-100000
Kontrol-gerçek adrese dönüşüm





Mantıksal veya Fiziksel Adres Uzayı

- Mantıksal bir adres uzayının ayrı bir fiziksel adres uzayına bağlanması düzgün bir hafıza yönetimi için önşarttır
 - **Mantıksal adres** (logical address) –CPU tarafından oluşturulur.
Sanal adres (virtual address) olarak da adlandırılır
 - **Fiziksel adres** (physical address) – hafıza birimi tarafından bilinen adrestir

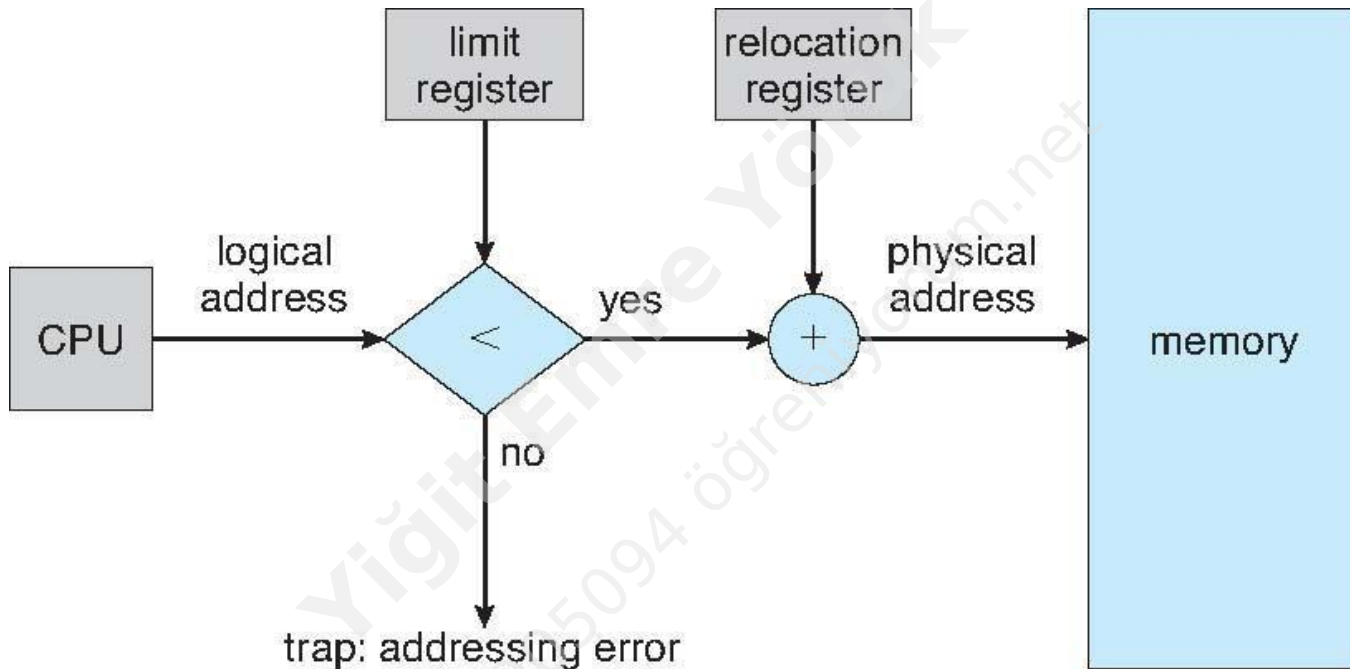
İşlemlerin adreslerine mantıksal adres (logical address)-hafızadaki adreslere fiziksel adres(physical address)

İşlemler gerçekte hangi adreste olduklarını - fiziksel adreslerini bilmezler.





Yeniden Yerleştirme ve Sınır Yazmaçları için Donanım Desteği





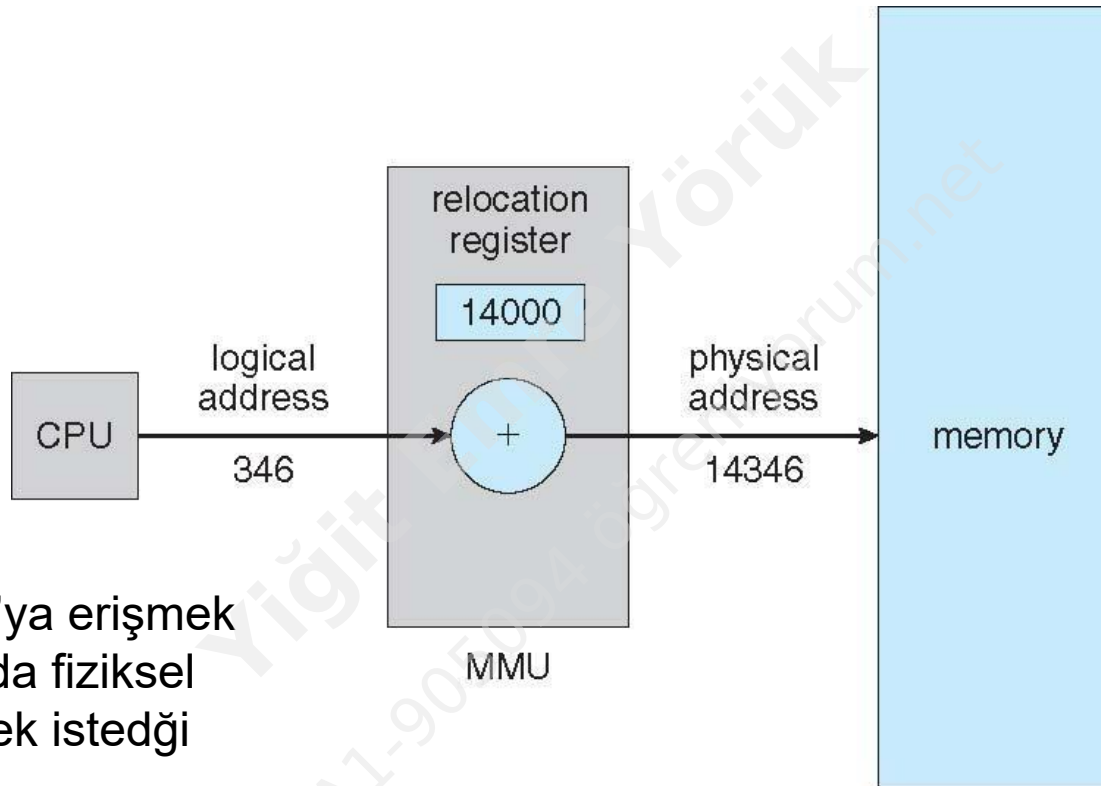
Hafıza Yönetim Birimi (MMU)

- Sanal adresleri fiziksel adreslere çeviren donanım
- MMU birimine kullanıcı programı tarafından gönderilen her adrese **yeniden yerleştirme yazmacındaki** (relocation register) değer eklenir
- Elde edilen adres hafızaya gönderilir
- Kullanıcı programları mantıksal adresleri kullanırlar; gerçek fiziksel adresleri *asla* görmezler





Yeniden Yerleştirme Yazmacını Kullanarak Dinamik Yeniden Yerleştirme



Process 346'ya erişmek istiyor. Aslında fiziksel olarak erişmek istediği 14346
İS bu mantıksal adresten fiziksel adrese dönüşümü sağlıyor





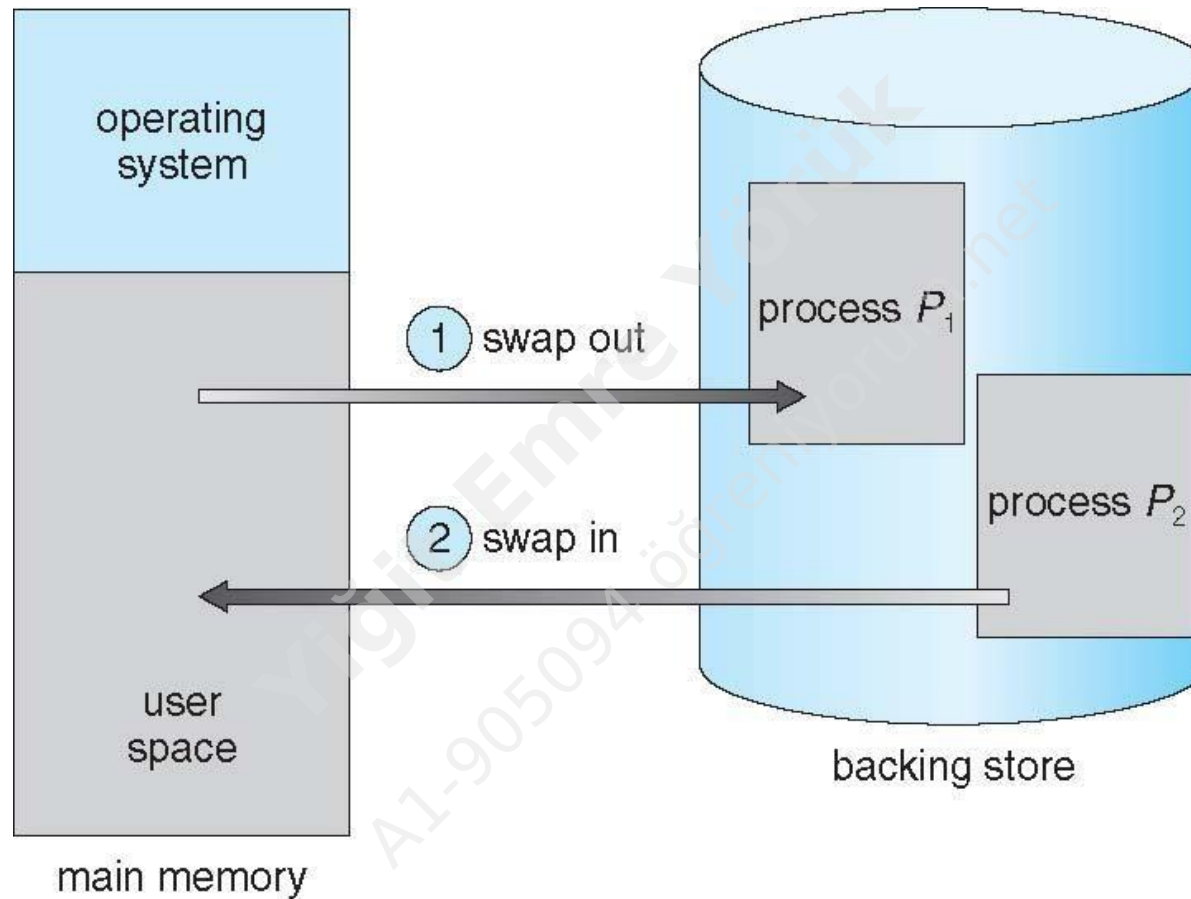
Yer Değiştirme (Swapping) (1/2)

- Bir işlem geçici olarak hafızadan çıkarılıp ikincil bir kayıt birimine Alınabilir/**virtual memory-disk**in belirli bir kısmını RAM olarak kullanma
 - Hafızada yer yoksa
- İşlem daha sonra çalışmaya devam etmek üzere yeniden ana belleğe alınabilir
- **Roll out, roll in** – öncelik tabanlı zamanlama algoritmaları tarafından kullanılan yer değiştirme varyasyonu
 - **Düşük öncelikli işlemler, daha yüksek öncelikli işlemlerin yüklenip** çalıştırılabilmesi için geçici olarak hafıza dışına alınır





Yer Değiştirme





Bitişik Bellek Yerleşimi

- Ana bellek genellikle iki bölüme ayrılır:
 - İşletim sistemi, genellikle kesinti vektörü ile birlikte hafızanın alt kısmında yer alır
 - Kullanıcı işlemleri, hafızanın üst kısmında yer alır
- Yeniden yerleştirme yazmaçları kullanıcı işlemlerini birbirinden korumak, işletim sistemi kodu ve verilerinin değiştirilmesini önlemek için kullanılır
 - **Taban yazmacı**, en küçük fiziksel adresin değerini tutar
 - **Sınır yazmacı**, mantıksal adreslerin sınır değerini tutar – tüm mantıksal adresler sınır yazmacında tutulan değerden daha küçüktür
 - **MMU** mantıksal adresleri *dinamik* olarak çevirir

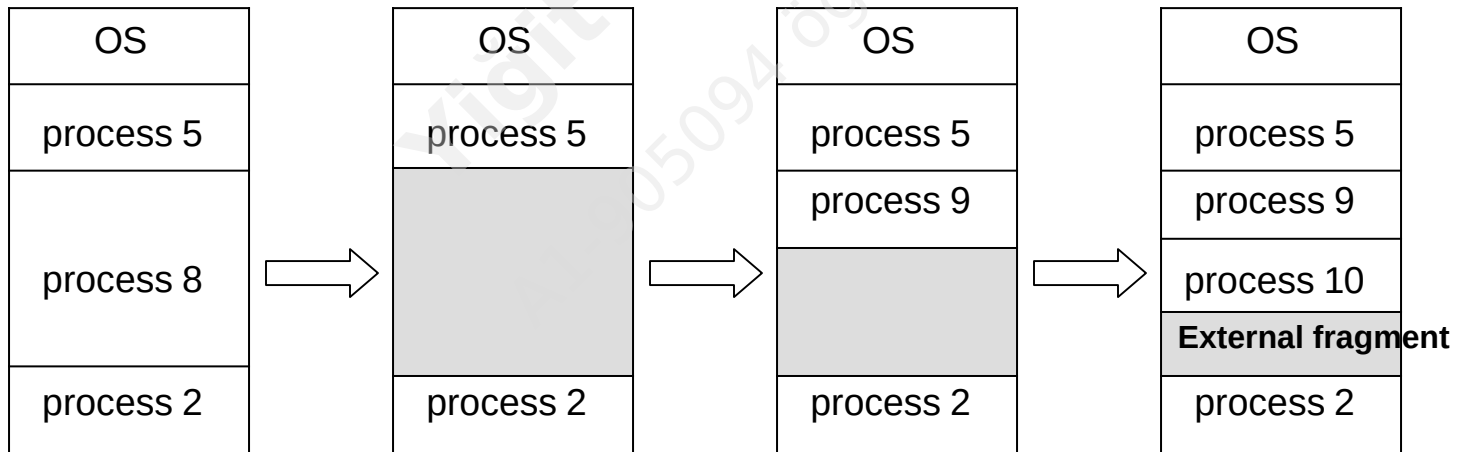




Bitişik Bellek Yerleşimi (Devam)

■ Çoklu-bölüm ayırımı

- Boşluk (Hole) – kullanılabilir bellek bloğu; hafıza üzerinde çeşitli boyutlarda boşluklar dağınık bir şekilde bulunur
- Bir işlem geldiğinde, bu işlemi tutabilecek kadar büyük bir hafıza boşluğuna yerleştirilir
- İşletim sistemi şu bilgileri tutup günceller:
a) işlemlere ayrılmış bölümler b) boş bölümler (boşluklar)





Dinamik Kayıt Birimi Ayırım Problemi

Bir boş bölümler listesi elimizdeyken, n boyutunda bir işlemi nasıl bir boşluğa atayabiliriz?

- **İlk-uyum (first-fit)**: yeterince büyük olan *ilk* boşluğa ata
- **En-iyi-uyum (best-fit)**: yeterince büyük olan *en küçük* boşluğa ata
 - boyuta göre sıralı değilse, tüm liste aranmalıdır
 - geriye en küçük boşluğu bırakır
- **En-kötü-uyum (worst-fit)**: mevcut *en büyük* boşluğa ata
 - tüm liste aranmalıdır
 - geriye en büyük boşluğu bırakır

İlk-uyum ve en-iyi-uyum, hız ve kayıt biriminin verimli kullanımı açısından en-kötü-uyum'a göre daha iyi sonuç verir





Parçalanma (Fragmentation) (1/2)

- **Dışsal Parçalanma (External Fragmentation)** – isteği karşılamak için hafıza alanı mevcut fakat bitişik değil
- **İçsel Parçalanma (Internal Fragmentation)** – işleme ayrılan hafıza gerekenden biraz fazla
 - boyut farkı işleme ayrılan hafıza alanında oluşuyor ve bu alan kullanılamıyor
 - İçsel parçalanma-bir process için gereğinden fazla hafıza ayrılıyorsa olur
 - Dışsal parçalanma process'i ekle, kaldır sonucunda meydana gelir

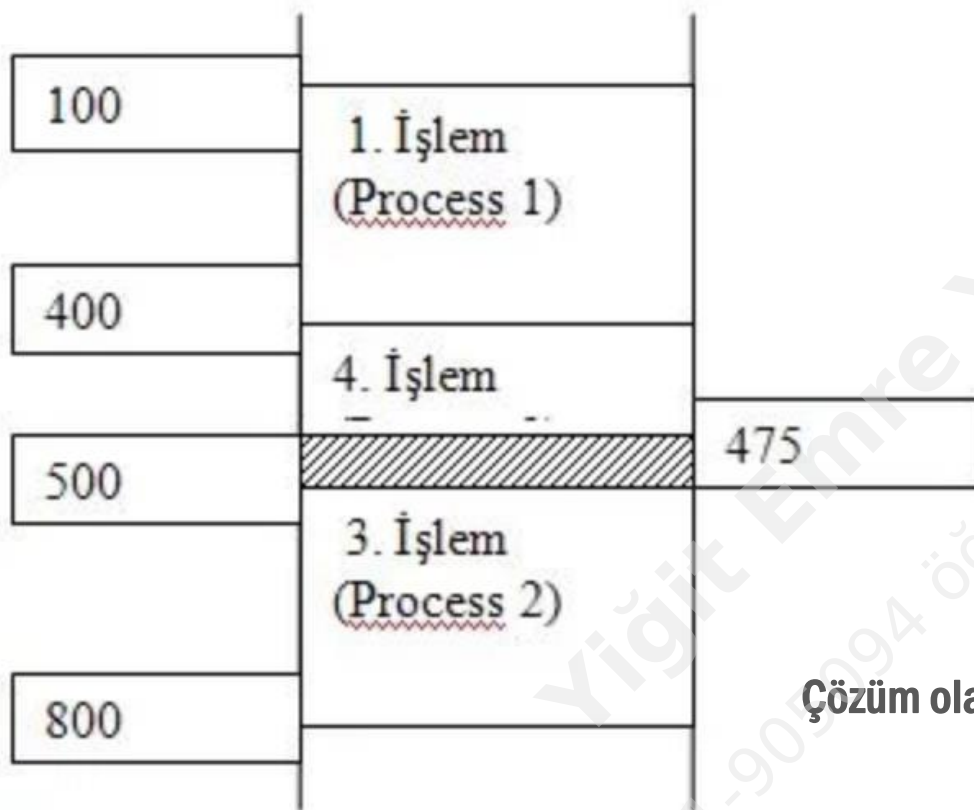




Parçalanma (Fragmentation) (2/2)

- Dışsal parçalanmayı **sıkıştırma (compaction)** ile azalt
 - Hafıza bloklarını, tüm boş blokları biraraya getirecek şekilde yeniden düzenle
 - I/O problemi
 - ▶ İşlemler I/O gerçekleştirirken hafızadaki yerlerini sabitle
 - ▶ I/O işlemlerini sadece işletim sistemine ait tampon bellekler üzerinde gerçekleştir
 - ▶ Çok maliyetlidir sıkıştırma-Bu sebeple yeni çözümler gerektirir





400-500 fiziksel adresleri
(physical address) arasında yer
alan 2. işlem (process 2)
kaldırılmış

P2 yerine 75 boyutunda 4. işlem

5 tane 25'lik external
fragmentasyon

Çözüm olarak Sayfalama (Paging)





Sayfalama (Paging) (1/2)

- Sayfalama, bir işleme ayrılan fiziksel adres uzayının **bitişik olmamasına** (noncontiguous) izin veren bir hafıza yönetim şeklidir
- Fiziksel hafıza sabit büyüklükte bloklara, **çerçevelere (frames)**, bölünür (belirlenen boyut 2'nin katları halindedir, 512 bayt ile 8,192 bayt arası)
- Mantıksal hafıza alanı da aynı boyutta bloklara, **sayfalara (pages)**, bölünür
- Tüm boş çerçeveler takip edilir: **çerçeve tablosu (frame table)**
- **n** sayfalık bir programı çalıştırmak için, n tane boş çerçevenin bulunması ve bu çerçevelere programın yüklenmesi gerekir
- Mantıksal adreslerin fiziksel adreslere dönüştürülmesi için bir **sayfa tablosuna (page table)** ihtiyaç duyulur





Adres Dönüşüm Mekanizması

- Sayfalamada boyut sabittir, parçalamada bölümler farklı boyutlara sahip olabilir.
- CPU tarafından üretilen adresle ikiye bölünür:
 - **Sayfa numarası (page number) (p)** – fiziksel hafızadaki her bir sayfanın temel adresini içeren sayfa tablosunun indeksi olarak kullanılır
 - **Sayfa ofseti (page offset) (d)** – temel adres ile birleştirilerek hafıza birimine gönderilecek fiziksel hafıza adresi elde edilir

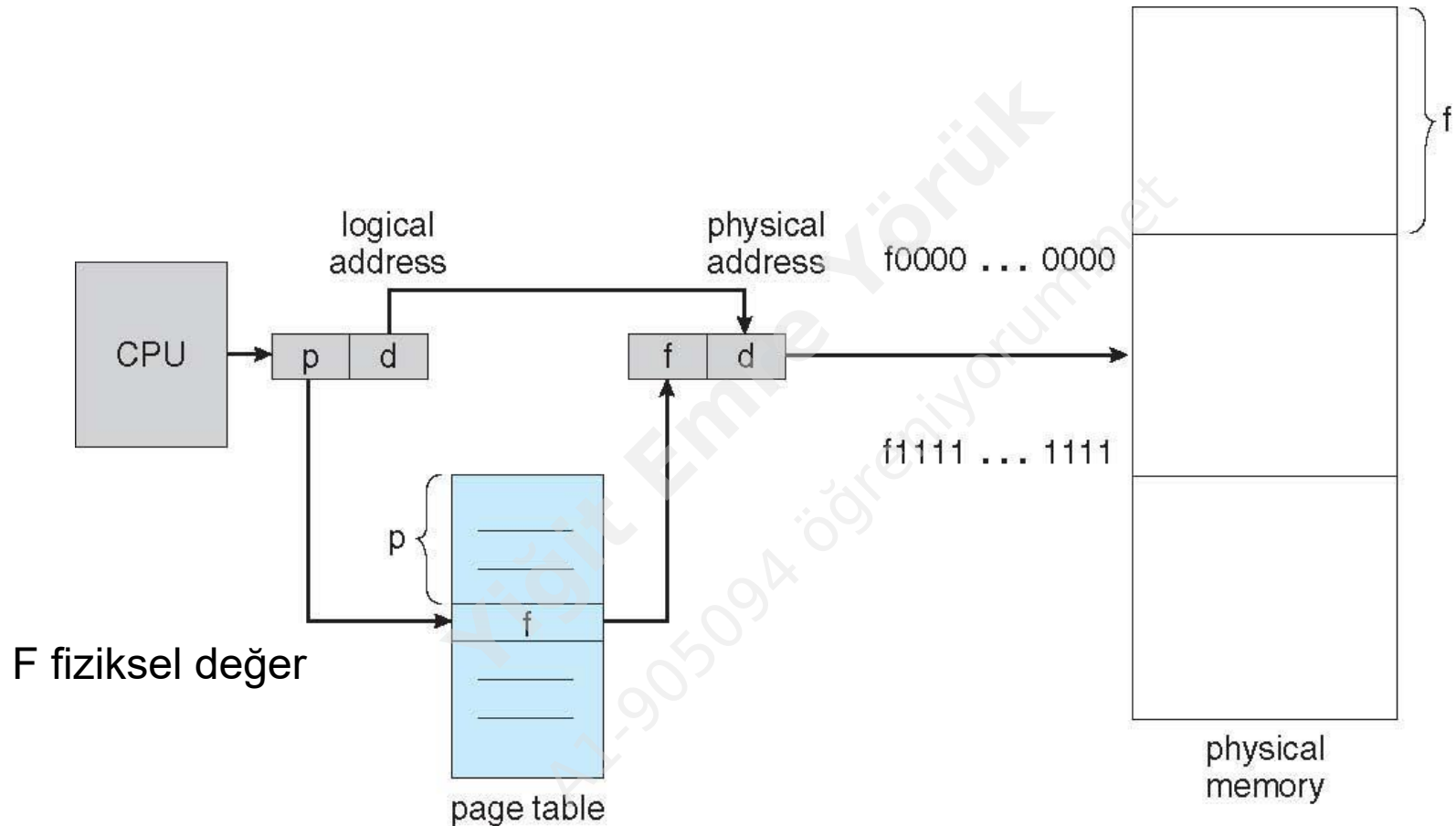
page number	page offset
p	d
$m - n$	

- 2^m mantıksal adres uzayı ve 2^n sayfa boyutu ile



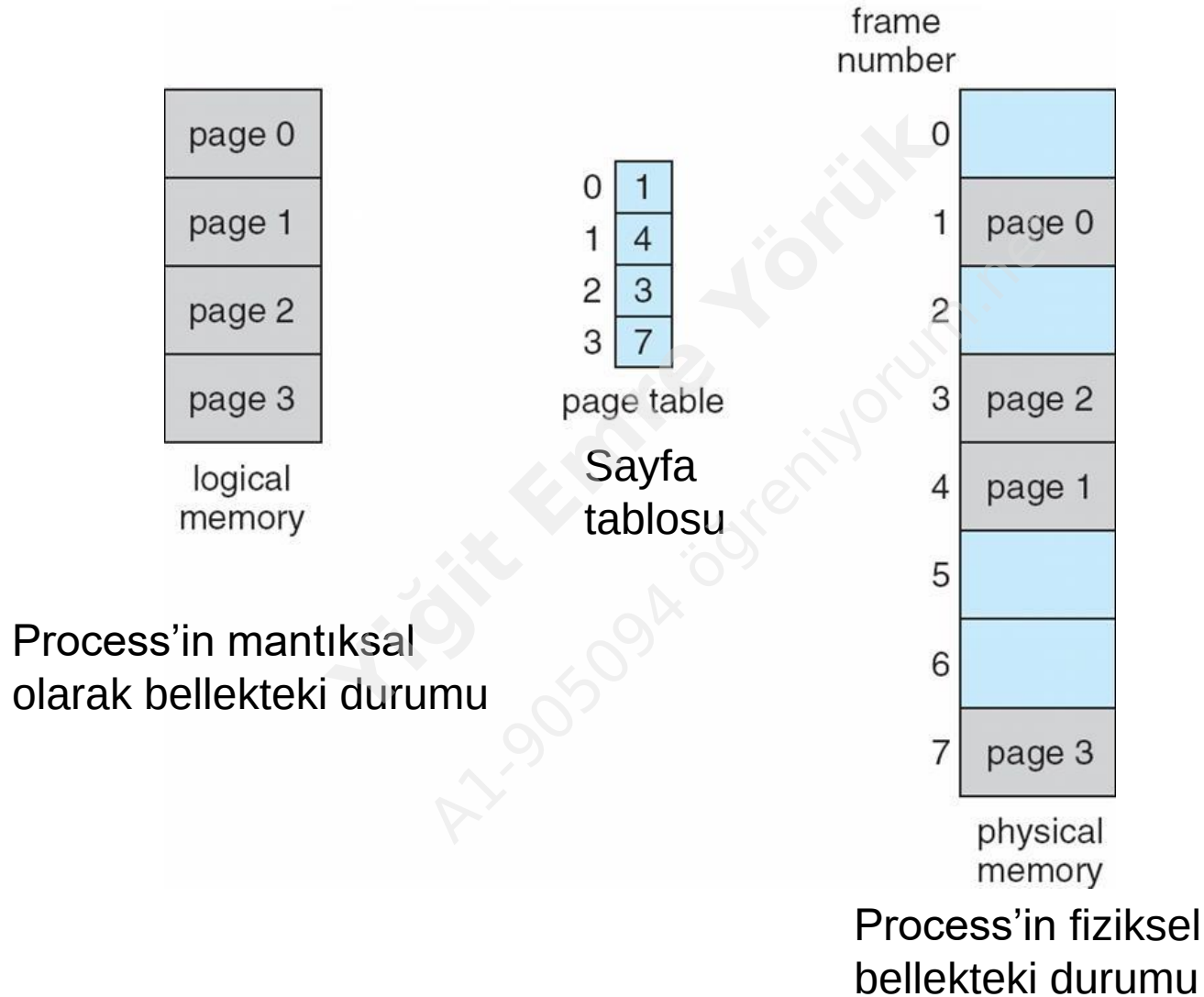


Sayfalama Donanımı





Mantıksal ve Fiziksel Hafızanın Sayfalama Modeli





Sayfalama Örneği

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

logical memory

0	5
1	6
2	1
3	2

page table

0		0
4	i j k l	1
8	m n o p	2
12		3
16		4
20	a b c d	5
24	e f g h	6
28		7

physical memory

32 bayt hafıza ve 4 baytlık sayfalar ($n=2$, $m=5$):
 $5-2=3$ / 8 sayfa





logical address 48 bit ,fiziksel adres 32 bitten oluşmaktadır bir page'in büyüklüğü 8 kb ise page table'in entry sayısı nedir?

$$\text{Mantıksal Adresleme Aralığı} = 2^{\text{Mantıksal Adres}} = 2^{48}$$

8 kb kaç bittir?

$$8 \times 8 \times 1024 = 2^{16}$$

$$48 - 16 = 32 \text{ sayfa var}$$





275 boyutundaki bir işlem aşağıdaki şekilde 3 sayfaya bölünecektir.

İşlem 1 (Process 1)	
Adres	Sayfa
0-100	1
100-200	2
200-275	3

Benzer şekilde 325 uzunluğuna sahip başka bir işlem de 4 parça

İşlem 2 (Process 2)	
Adres	Sayfa
0-100	1
100-200	2
200-300	3
300-325	4

Hafıza (RAM)		
Adres	Çerçeve	Sayfa
0-100	1	İ1-1
100-200	2	İ1-2
200-300	3	İ1-3
300-400	4	İ2-1
400-500	5	İ2-2
500-600	6	İ2-3
600-700	7	İ2-4
700-800	8	
800-900	9	
900-1000	10	





Sayfa Tablosu	
Sayfa	Çerçeve
İ1-1	1
İ1-2	2
İ1-3	3
İ2-1	4
İ2-2	5
İ2-3	6
İ2-4	7

Mantıksal adresi 327 olan ikinci işlemin fiziksel adresini bulalım.

1. Hangi sayfada?
 $327/100=3$
2. $3+1=4$. sayfada
3. 4. sayfa kaçinci çerçevede? (7)
4. 7. çerçevenin başlangıç değeri 600
5. Mantıksal adres offset'i?
27
6. Fiziksel adres= $600+27=627$





Logical adres=320

Page size=150

Frame size=150

Sayfa	Çerçeve
1	8
2	2
3	7
4	5
5	4

1
2
3
4
5

Process

1
2-Page 2
3
4-Page 5
5-Page 4
6
7-Page 3
8-Page 1

RAM

Fiziksel adresini hesaplayalım:





Logical adres=320

Page size=150

Frame size=150

Sayfa	Çerçeve
1	8
2	2
3	7
4	5
5	4

1
2
3
4
5

Process

1
2-Page 2
3
4-Page 5
5-Page 4
6
7-Page 3
8-Page 1

RAM

Fiziksel adresini hesaplayalım:

Sayfa numarası: $320/150=2+1=3$. sayfada

3. Sayfa 7. frame'de

7. frame'in başlangıcı 900

Offset: 20

FA: $900+20=920$





Logical adres=650

Page size=150

Frame size=150

Sayfa	Çerçeve
1	8
2	2
3	7
4	5
5	4

1
2
3
4
5

Process

1
2-Page 2
3
4-Page 5
5-Page 4
6
7-Page 3
8-Page 1

RAM

Fiziksel adresini hesaplayalım:

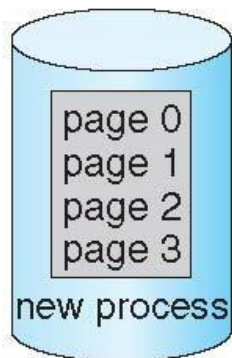




Boş Çerçeveler (Free Frames)

free-frame list

14
13
18
20
15

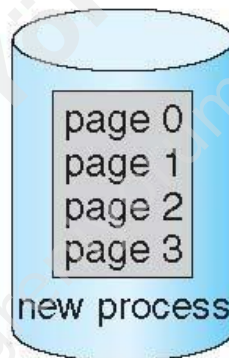


(a)

Ayrım öncesi

free-frame list

15



0	14
1	13
2	18
3	20

new-process page table

(b)

Ayrım sonrası





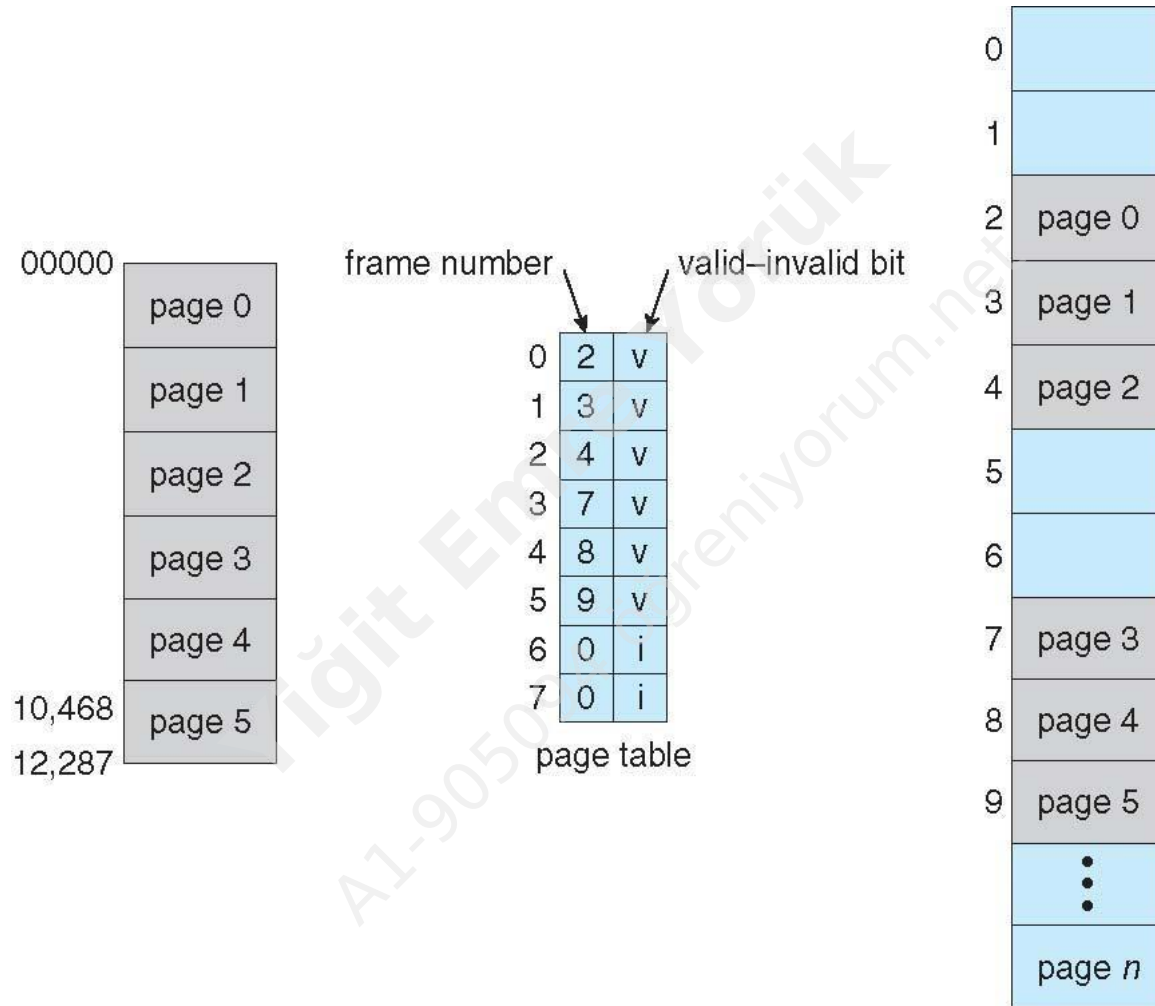
Hafıza Koruma

- Hafıza koruması, her bir tablo girişi ile bir koruma bitinin ilişkilendirilmesi ile mümkündür
- **Gçerli-Geçersiz biti (Valid-invalid bit):**
 - “geçerli” (valid), ilişkili sayfanın işlemin mantıksal adres uzayında olduğunu ve erişim hakkının geçerli olduğunu gösterir
 - “geçersiz” (invalid), ilişkili sayfanın işlemin mantıksal adres uzayında olmadığını gösterir





Sayfa Tablosundaki Geçerli (v) veya Geçersiz (i) Bitler





Paylaşımlı Sayfalar

■ Kod paylaşımı

- Sadece okunabilir olan kodun bir kopyası işlemler arasında paylaştırılır (örn: metin editörleri, derleyiciler, pencere sistemleri)
- **Paylaşılan kod, tüm işlemlerin mantıksal adres uzaylarında aynı konumda bulunmalıdır**

■ Özel kod ve veri

- Her bir işlem kod ve verilerin ayrı bir kopyasını tutarlar
- Özel kod ve veriye ait sayfalar mantıksal adres uzayının herhangi bir yerinde bulunabilir





Paylaşımlı Sayfa Örneği

